NAV4RAIL

Decisions d'architecture

Document de reference pour les choix techniques
du pipeline de fine-tuning de Behavior Trees

Equipe NAV4RAIL — SNCF

3 mars 2026

Ce document couvre cinq axes de decision : jeu de donnees synthetique (proxy), rejet du RAG, securite et watchdog, fine-tuning LoRA, et evaluation des Behavior Trees.

Table des matières

1 Introduction **3**
1.1 Glossaire des termes et acronymes . . . 3

2 Axe 1 — Jeu de donnees synthetique (proxy) **5**
2.1 Contexte . . . 5
2.2 Decision . . . 5
2.3 Methode de generation . . . 5
2.3.1 Decomposition en blocs fonctionnels . . . 5
2.3.2 Composition en templates . . . 5
2.3.3 Randomisation des missions . . . 6
2.4 Evolution du jeu de donnees . . . 6
2.5 Les 8 categories du dataset v4 scaled (2 000 exemples) . . . 7
2.6 Justification et limites . . . 7

3 Axe 2 — RAG : analyse et rejet motive **8**
3.1 Contexte . . . 8
3.2 Decision . . . 8
3.3 Analyse quantitative . . . 8
3.4 Alternatives ecartees . . . 8
3.5 Architecture retenue . . . 9

4 Axe 3 — Securite et watchdog : defense en profondeur **10**
4.1 Contexte . . . 10
4.2 Decision . . . 10
4.3 Vue d'ensemble du pipeline . . . 10
4.4 Couche 1 : Decodage contraint (niveau token) . . . 10
4.5 Couche 2 : Extraction XML (post-traitement) . . . 11
4.6 Couche 3 : Validateur a trois niveaux . . . 11

5 Axe 4 — Fine-tuning LoRA : choix techniques **12**
5.1 Contexte . . . 12
5.2 Decision : QLoRA 4-bit NF4 . . . 12
5.3 Parametres d'entrainement . . . 12
5.4 Importance du masquage de la perte . . . 12
5.5 Choix du modele . . . 12
5.6 Resultats . . . 13

6 Axe 5 — Evaluation : syntaxe, structure, semantique **14**
6.1 Contexte . . . 14
6.2 Decision . . . 14
6.3 Niveau L1 — Syntaxique (bloquant) . . . 14
6.4 Niveau L2 — Structurel (bloquant) . . . 15

6.5 Niveau L3 — Semantique (avertissements) 15
6.6 Protocole de test 15

7 Tableau recapitulatif **17**

8 Catalogue des 27 skills NAV4RAIL **17**

1 Introduction

Le projet **NAV4RAIL** vise a permettre a un operateur de decrire une mission ferroviaire en langage naturel, puis de generer automatiquement un **Behavior Tree** (arbre de comportement, ou **BT**) au format XML compatible avec la bibliotheque **BehaviorTree.CPP v4**. Ces BT commandent un robot de maintenance sur le reseau ferre de la SNCF.

Le pipeline repose sur le **fine-tuning** (ajustement fin) d'un modele de langage (**LLM**, pour *Large Language Model* — grand modele de langage) avec la technique **QLoRA** (une methode pour entrainer seulement une petite fraction des parametres du modele, tout en gardant le reste gele et compresse en 4 bits).

Ce document consolide les cinq decisions architecturales majeures du projet :

1. **Axe 1 — Jeu de donnees synthetique (proxy)** : comment 2 000 exemples d'entrainement ont ete generes a partir d'un seul exemple XML reel.
2. **Axe 2 — RAG : analyse et rejet motive** : pourquoi la recherche augmentee par recuperation de documents (*RAG*) a ete ecartee.
3. **Axe 3 — Securite et watchdog** : l'architecture de defense en profondeur a trois couches.
4. **Axe 4 — Fine-tuning LoRA** : les choix techniques d'entrainement.
5. **Axe 5 — Evaluation** : le validateur a trois niveaux (syntaxe, structure, semantique).

1.1 Glossaire des termes et acronymes

BT — **Behavior Tree** — arbre de comportement. Structure arborescente qui decompose une tache complexe en sous-taches (noeuds). Utilise en robotique et dans les jeux video.

BehaviorTree.CPP — Bibliotheque C++ de reference pour executer des BT. Le format v4 est le standard XML utilise dans NAV4RAIL.

LLM — **Large Language Model** — grand modele de langage. Reseau de neurones pre-entraine sur de vastes corpus de texte, capable de generer du texte (ici, du XML).

Fine-tuning — Ajustement fin : on reprend un LLM pre-entraine et on l'entraine sur un petit jeu de donnees specialise pour qu'il apprenne une tache precise.

LoRA — **Low-Rank Adaptation** — adaptation de rang faible. Technique qui ajoute de petites matrices entrainables a cote des poids geles du modele, reduisant considerablement la memoire requise.

QLoRA — **Quantized LoRA** — LoRA quantifie. Variante de LoRA ou les poids du modele de base sont comprimes en 4 bits (*NF4*, un format de quantification optimise pour les distributions normales des poids). Permet d'entrainer un modele de 7 milliards de parametres sur un GPU de 16 Go.

NF4 — **NormalFloat 4-bit** — format de quantification 4 bits concu pour les poids de reseaux de neurones, dont la distribution suit approximativement une loi

normale.

RAG **Retrieval-Augmented Generation** — generation augmentee par la recuperation. Architecture ou le modele consulte une base de connaissances externe avant de generer sa reponse.

GBNF **GGML BNF** — format de grammaire (derive de la forme Backus-Naur) utilise pour contraindre la generation de texte token par token. Chaque token genere doit respecter la grammaire.

SLURM Gestionnaire de taches pour clusters de calcul.

VRAM Memoire video du GPU. Facteur limitant principal pour l'entrainement de LLM.

Proxy (jeu de donnees)
Ici, *proxy* designe le jeu de donnees synthetique qui *remplace* (sert de substitut a) des donnees reelles annotees a la main.

Skill Competence atomique du robot (ex. : `<LoadMission/>`, `<Move/>`). Chaque skill est un noeud feuille dans le BT.

Sequence Noeud de controle du BT : execute ses enfants de gauche a droite, s'arrete au premier echec.

Fallback Noeud de controle du BT : essaie ses enfants de gauche a droite, s'arrete au premier succes. Utilise pour les boucles et la gestion d'erreurs.

2 Axe 1 — Jeu de donnees synthetique (proxy)

2.1 Contexte

Le fine-tuning d'un LLM necessite des milliers de paires *(mission, BT attendu)*. Or, dans le domaine ferroviaire, il n'existe pas de corpus annote de Behavior Trees. Annoter manuellement 2 000 exemples serait extremement couteux et sujet a erreurs humaines.

La question centrale etait : **comment obtenir suffisamment de donnees d'entrainement a partir d'un seul exemple XML reel fourni par l'equipe SNCF ?**

2.2 Decision

Generer un **jeu de donnees synthetique** (le *proxy*) par **composition de patrons** (templates). Un seul exemple reel de Behavior Tree a ete decompose en blocs fonctionnels reutilisables, puis recombine de multiples facons pour produire 2 000 paires d'entrainement.

2.3 Methode de generation

La generation repose sur trois piliers :

2.3.1 Decomposition en blocs fonctionnels

L'exemple XML reel a ete analyse et decompose en **blocs reutilisables** :

- **Blocs de preparation** : chargement de mission, validation de structure, calcul d'itineraire (`<LoadMission/>` → `<MissionStructureValid/>` → `<ProjectPointOnNetwork/>` → `<CreatePath/>` → `<AgregatePath/>`).
- **Boucles de mouvement** : schema `Fallback(MissionTerminated | Sequence(etape))` — le robot repete une etape tant que la mission n'est pas terminee.
- **Blocs d'inspection** : mesure, analyse, controle qualite avec branchement correctif si la qualite est insuffisante.
- **Blocs correctifs** : detection de defauts → generation de sous-sequence corrective → insertion.

Ces blocs sont implementes sous forme de fonctions Python dans le script `generate_dataset_v4.py`, chacune retournant un fragment XML valide.

2.3.2 Composition en templates

Les blocs sont assembles en **templates complets** (environ 50 fonctions generatrices) : chaque template represente un type de mission complet. Par exemple :

- `xml_nav_simple()` : preparation + boucle de mouvement + arret final.
- `xml_inspect_corrective()` : preparation + boucle d'inspection avec controle qualite et sequence corrective.
- `xml_simulation_nav()` : verification de simulation + navigation complete.

— `xml_complex_nav_measure_return()` : navigation multi-segments + mesures + retour au depot.

2.3.3 Randomisation des missions

Chaque exemple est unique grace a la **variation linguistique** des descriptions de mission. Le generateur combine aleatoirement :

— Des **verbes** : « Deplace-toi », « Va », « Rejoins », « Navigue jusqu'a »
— Des **cibles** : « le depot principal », « le poste de maintenance », « la zone de stationnement »
— Des **objets d'inspection** : « la voie », « les rails », « les aiguillages »
— Des **localisations** : « entre le km 15 et le km 28 », « section A », « tunnel nord »

2.4 Evolution du jeu de donnees

Le dataset a evolue en quatre versions :

Table 1 – Evolution du jeu de donnees proxy

Version	Exemples	Skills	Templates	Apport principal
v1	100	8 proxy	∼10	Premier prototype, indentation non uniforme
v2	500	8 proxy	∼10	5 categories, indentation 2 espaces uniforme, XML builder
v3	550	8 proxy	∼10	+50 exemples « inspection securisee » (Fallback + CheckObstacle)
v4	550	**27 reels**	∼28	Remplacement des skills proxy par les 27 skills reels NAV4RAIL
v4 scaled	**2 000**	**27 reels**	**∼50**	8 categories, ∼50 templates, patterns avances

Le passage v3 → v4 est la transition cle : les 8 skills « proxy » simplifies (ex. : `GetMission`, `CalculatePath`) ont ete remplaces par les 27 skills reels du systeme NAV4RAIL (ex. : `<LoadMission/>`, `<MissionStructureValid/>`, `<ProjectPointOnNetwork/>`, etc.).

La correspondance entre skills proxy et skills reels est documentee dans `SKILLS_CATALOG.md`. Par exemple :

Table 2 – Correspondance skills proxy → skills reels (extrait)

Skill proxy	Skills reels correspondants
`GetMission`	`<LoadMission/>` + `<MissionStructureValid/>`
`CalculatePath`	`<ProjectPointOnNetwork/>` + `<CreatePath/>` + `<AgregatePath/>`
`Move`	`<PassMotionParameters/>` + `<Move/>` (ou `<MoveAndStop/>`)
`ManageMeasurement`	`<ManageMeasurements/>` + `<AnalyseMeasurements/>`
`Stop`	`<MissionTerminated/>` (condition) + `<MoveAndStop/>` (action)

Table 3 – Repartition par categorie dans le dataset v4 scaled

#	Categorie	Exemples	Particularite
1	Navigation simple	350	Preparation + mouvement + arret
2	Navigation avec autorisation	150	+ `<IsRobotPoseProjectionActive/>`
3	Inspection de voie	350	Boucle mesure/analyse/qualite
4	Inspection avec corrective	200	+ sequence corrective si defaut
5	Mesures simples	150	Mouvement + deceleration + mesure
6	Navigation securisee	150	Deceleration + signalisation
7	Missions complexes	400	Combinaisons multi-phases
8	Simulation	250	Prefixe `<SimulationStarted/>`

2.5 Les 8 categories du dataset v4 scaled (2 000 exemples)

2.6 Justification et limites

Pourquoi cette approche fonctionne :

— Les BT ont une structure **compositionnelle** : un nombre fini de blocs se combinent pour couvrir tous les cas d'usage.

— La randomisation linguistique assure que le modele apprend a *comprendre l'intention* de la mission, pas a memoriser des formulations precises.

— Les 27 skills sont **sans parametre** (hormis l'attribut `name`) : le modele doit seulement choisir *quels* skills utiliser et dans *quel ordre*, pas deviner des valeurs numeriques.

Quand reconsiderer :

— Si les skills deviennent **parametres** (ex. : `sensor="lidar_3d"`, `target_km="8.5"`), la generation synthetique ne suffira plus — il faudra des exemples reels annotes ou un mecanisme RAG.

— Si le nombre de skills depasse ∼50, la combinatoire des templates deviendra difficile a maintenir manuellement.

References : `generate_dataset_v4.py`, `SKILLS_CATALOG.md`.

3 Axe 2 — RAG : analyse et rejet motive

3.1 Contexte

Le **RAG** (*Retrieval-Augmented Generation*, generation augmentee par la recuperation) est une architecture populaire ou, avant de generer une reponse, le modele consulte une base de connaissances externe pour recuperer les informations pertinentes. La question etait : **faut-il recuperer dynamiquement les skills pertinents a chaque mission, plutot que de tous les injecter dans le prompt ?**

3.2 Decision

Le RAG a ete **rejete**. A la place, tous les 27 skills sont injectes *statiquement* dans le prompt systeme, et le decodage contraint (GBNF) empeche structurellement toute hallucination de skill.

3.3 Analyse quantitative

Quatre criteres de decision ont ete evalues :

Table 4 – Criteres de decision Fine-tuning vs. RAG

Critere	Valeur reelle	Seuil RAG	Decision
Nombre de skills	27	> 50	Fine-tuning
Parametres par skill	1 (`name`)	> 1	Fine-tuning
Frequence de mise a jour	Stable	$> 1\times$/mois	Fine-tuning
Budget token (27 skills)	$\sim$270 tok ($\sim$7 %)	> 50 %	Fine-tuning

Le calcul est le suivant : 27 skills $\times$ $\sim$10 tokens par skill $\approx$ 270 tokens, ce qui occupe seulement **7 %** de la fenetre de contexte de 4 096 tokens de Mistral-7B. Il reste largement assez de place pour le prompt systeme ($\sim$200 tokens), la description de mission ($\sim$50 tokens), et le BT genere ($\sim$500 tokens).

3.4 Alternatives ecartees

Trois alternatives au RAG ont ete examinees et ecartees :

Option A — Injection selective par regles Selection des skills par mots-cles (*« si la mission contient 'inspecte', injecter les skills d'inspection »*). Ecartee car fragile face aux missions ambigues ou multi-categories (ex. : « inspecte et reviens au depot »).

Option B — Grammaire GBNF etendue Ajouter de nouveaux skills a la grammaire GBNF au fil du temps. Deja partiellement implementee, mais la maintenance manuelle de la grammaire deviendrait penible au-dela de $\sim$80 skills.

Option C — Recuperation d'exemples few-shot Pour chaque mission de test, recuperer les 2–3 exemples d'entrainement les plus proches et les inclure dans le prompt. Ecartee car cela augmente la longueur du prompt sans apporter de gain mesurable (le modele fine-tune genere deja des BT corrects sans exemples).

3.5 Architecture retenue

L'architecture adoptee combine deux mecanismes :

1. **Prompt statique** : le catalogue complet des 27 skills est injecte dans le prompt systeme (∼270 tokens, 7 % du contexte).
2. **Decodage contraint** : la grammaire GBNF agit comme un « RAG au niveau du decodage » — a chaque token genere, seuls les tokens correspondant a des skills valides sont autorises.

Quand reconsiderer :

— Si le nombre de skills depasse ∼50 (le budget token depasserait 500 tokens, soit ∼12 % du contexte — toujours acceptable, mais la tendance est a surveiller).

— Si les skills deviennent fortement parametres (parametres derives de la mission, ex. : kilometres, capteurs, seuils).

— Si la frequence de mise a jour depasse 1×/mois (chaque modification necessiterait un re-fine-tuning).

References : `RAG_ARCHITECTURE.md` (analyse complete), `SKILLS_CATALOG.md`, `nav4rail_grammar.py`.

4 Axe 3 — Securite et watchdog : defense en profondeur

4.1 Contexte

Les Behavior Trees generes par le LLM commandent un **robot reel** sur le reseau ferre. Un skill hallucine, un noeud manquant ou un ordre d'execution incorrect pourrait entrainer un comportement dangereux. La strategie de securite doit etre multi-couche : si une couche laisse passer une erreur, la suivante doit la detecter.

4.2 Decision

Architecture de **defense en profondeur a trois couches** :

1. **Couche 1** — Decodage contraint (niveau token)
2. **Couche 2** — Extraction XML (post-traitement)
3. **Couche 3** — Validateur a trois niveaux (verification complete)

4.3 Vue d'ensemble du pipeline

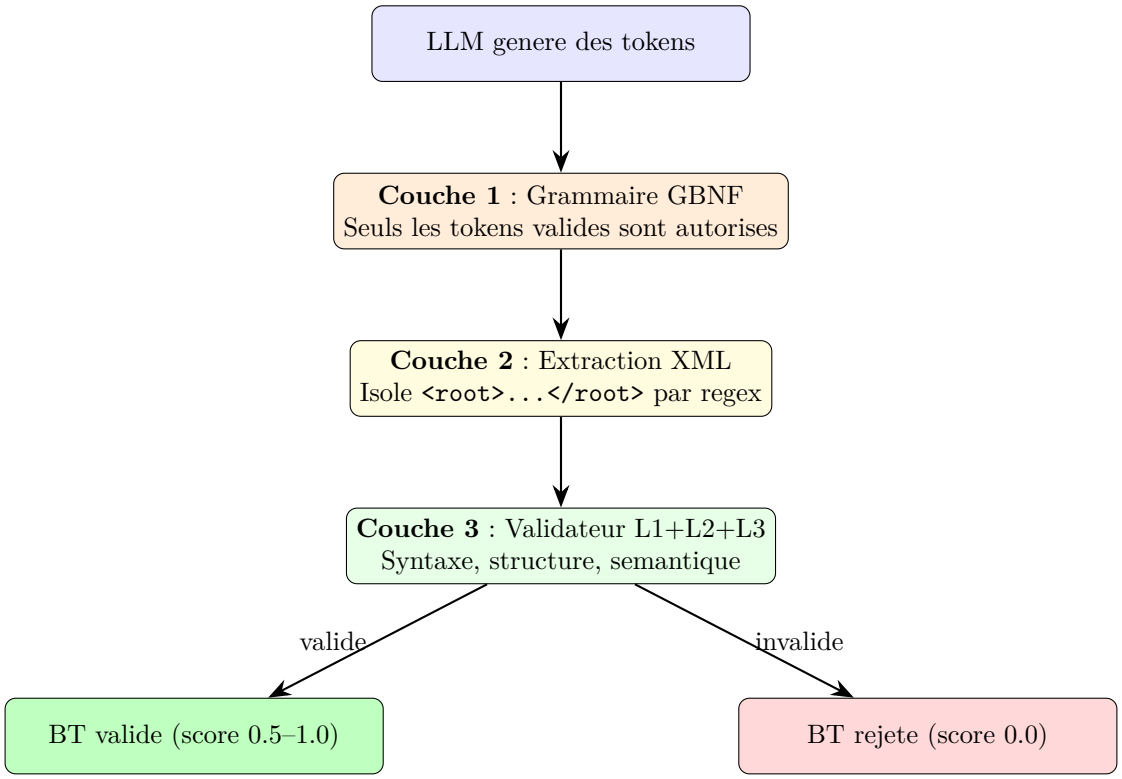

FIGURE 1 – Pipeline de defense en profondeur a trois couches

4.4 Couche 1 : Decodage contraint (niveau token)

Le decodage contraint empeche le LLM de generer des tokens qui violeraient la grammaire. A **chaque etape de generation** :

1. Le modele propose une distribution de probabilites sur tous les tokens possibles.

2. La fonction `prefix_allowed_tokens_fn` verifie quels tokens sont compatibles avec la grammaire GBNF.
3. Les tokens **incompatibles** recoivent un score de $-\infty$ (ils ne seront jamais choisis).
4. Le modele choisit parmi les tokens **restants** uniquement.

Implementation : Le fichier `nav4rail_grammar.py` definit :

— `NAV4RAIL_GBNF` : la grammaire au format GBNF, listant explicitement les 27 noms de skills autorises.

— `get_prefix_fn(tokenizer)` : cree la fonction de contrainte en utilisant la bibliotheque *lm-format-enforcer*[1], qui convertit un `RegexParser` en `prefix_allowed_tokens_fn`.

Garantie : *Zero hallucination de nom de skill.* Il est **structurellement impossible** pour le modele de generer un tag XML qui ne soit pas dans le catalogue des 27 skills.

Limite : La grammaire GBNF ne peut pas verifier l'equilibrage des balises (propriete *context-sensitive*, hors de portee d'une grammaire reguliere). Cette verification est deleguee a la Couche 3.

4.5 Couche 2 : Extraction XML (post-traitement)

Apres la generation, le texte brut peut contenir des artefacts (texte avant ou apres le XML, repetitions). La fonction `_extract_xml()` (`finetune_lora_xml.py`, lignes 275–284) isole le bloc XML valide :

```
pattern = r"(<root\b.*?</root>)"
match = re.search(pattern, raw_output, re.DOTALL)
xml_str = match.group(1) if match else raw_output
```

Cette etape est simple mais essentielle : elle garantit que seul le contenu entre `<root>` et `</root>` est transmis au validateur.

4.6 Couche 3 : Validateur a trois niveaux

Le fichier `validate_bt.py` implemente un validateur en trois niveaux, detaille dans la section 6.

Scoring :

— **1.0** : parfait (les 3 niveaux passes, aucun avertissement).

— **0.5–0.9** : valide avec avertissements (-0.1 par avertissement, plancher a 0.5).

— **0.0** : invalide (erreur bloquante au niveau L1 ou L2).

References : `nav4rail_grammar.py`, `validate_bt.py`, `finetune_lora_xml.py` (lignes 275–284).

1. *lm-format-enforcer* : une bibliotheque Python qui transforme une expression reguliere ou une grammaire GBNF en contrainte token par token compatible avec Hugging Face `model.generate()`.

5 Axe 4 — Fine-tuning LoRA : choix techniques

5.1 Contexte

L'objectif est d'entrainer le modele **Mistral-7B-Instruct-v0.2** (7,3 milliards de parametres) sur un cluster equipe de GPU **Tesla P100** (16 Go de VRAM) et **RTX 3090** (24 Go de VRAM).

Probleme : un fine-tuning complet (tous les parametres) de Mistral-7B necessite environ 56 Go de VRAM — c'est-a-dire 3.5× la capacite d'un P100. La solution est **QLoRA**.

5.2 Decision : QLoRA 4-bit NF4

Au lieu de modifier les 7,3 milliards de parametres, QLoRA :

1. **Quantifie** les poids du modele de base en 4 bits NF4 (poids **geles**, non modifies pendant l'entrainement).
2. **Ajoute** de petites matrices entrainables (*adaptateurs LoRA*) a cote des poids geles.

La formule de sortie effective est :

$$\mathbf{y} = \underbrace{W_0^{\text{4-bit, gele}}}_{\text{modele de base}} \cdot \mathbf{x} + \underbrace{\frac{\alpha}{r} \cdot B \cdot A}_{\text{adaptateur LoRA}} \cdot \mathbf{x} \tag{1}$$

ou W_0 est la matrice de poids originale (gelee en 4 bits), A et B sont les petites matrices entrainables de rang r, et α est le facteur d'echelle.

5.3 Parametres d'entrainement

5.4 Importance du masquage de la perte

Le `DataCollatorForCompletionOnlyLM` est un choix crucial : il masque la perte (*loss*) sur toute la partie *instruction* du prompt (avant le token `[/INST]`). Le modele n'apprend **que** a generer le XML, pas a repeter le prompt systeme ou la description de mission.

Impact mesure : convergence 2× plus rapide. Sans ce masquage, la perte d'evaluation (*eval_loss*) stagnait a ∼0.05 ; avec le masquage, elle descend a 0.017 des la premiere version.

5.5 Choix du modele

Mistral-7B-Instruct-v0.2 Modele principal. Format d'instruction natif `[INST]...[/INST]`, bonnes performances en generation structuree. 7,3 milliards de parametres.

TinyLlama-1.1B-Chat Modele de reference (*baseline*). 1,1 milliard de parametres, 6 minutes d'entrainement. Utile pour valider le pipeline avant de lancer un entrainement long sur Mistral.

Table 5 – Configuration QLoRA pour Mistral-7B

Parametre	**Valeur**	**Explication**
Quantification	4-bit NF4 + double quant	Poids comprimes en 4 bits, seconde quantification pour les constantes
Rang LoRA (r)	16	Dimension des matrices A et B — compromis entre expressivite et taille
Alpha LoRA (α)	32	Facteur d'echelle ($\alpha/r = 2$), amplifie l'influence des adaptateurs
Modules cibles	7 (q, k, v, o, gate, up, down)	Toutes les projections d'attention + couches MLP
Params. entrainables	42 M (0.58 % de 7.3 B)	Seule cette fraction est modifiee pendant l'entrainement
Optimiseur	`paged_adamw_8bit`	AdamW en 8 bits avec pagination memoire (reduit la VRAM)
Planificateur LR	Cosine + 5 % warmup	Taux d'apprentissage decroit en cosinus apres un echauffement
Taux d'apprentissage	2×10^{-4}	Valeur standard pour QLoRA
Masquage de la perte	`DataCollatorForCompletionOnlyLM`	La perte n'est calculee *que* sur la reponse XML (apres le token `[/INST]`), pas sur le prompt
Taille de batch effective	64 (4 × 16)	4 exemples par GPU × 16 accumulations de gradient
Longueur max sequence	1 536 tokens	Couvre le prompt + BT genere (augmente depuis 1 024 en v4)
VRAM utilisee	∼5.0 Go / 15.9 Go (P100)	∼31 % de la capacite

5.6 Resultats

Observation cle : le passage aux 27 skills reels (v4) a ameliore le score semantique de 0.98 a **1.00** malgre la complexite accrue, car les patterns des skills reels sont plus reguliers que ceux des 8 skills proxy simplifies.

References : `finetune_lora_xml.py` (lignes 40–64 pour la configuration), `generate_dataset_v4.py`, `RESULTS.md`.

Table 6 – Evolution des performances par version du dataset

Configuration	Eval loss	Score L3	Warnings	Temps	GPU
TinyLlama, 100 ex.	0.118	n/a	—	6 min	P100
Mistral, 100 ex.	0.017	n/a	—	25 min	P100
Mistral, 500 ex. (v2)	0.010	0.97	3/10	140 min	P100
Mistral, 550 ex. (v3)	0.011	0.98	2/10	249 min	P100
Mistral, 550 ex. (v4)	**0.0035**	**1.00**	**0/10**	561 min	P100
Mistral, 2 000 ex. (v4 sc.)	0.0048	**1.00**	**0/15**	684 min	RTX 3090

6 Axe 5 — Evaluation : syntaxe, structure, semantique

6.1 Contexte

Un simple *parse* XML (verification syntaxique) est insuffisant : un XML syntaxiquement correct peut representer un BT structurellement incoherent ou semantiquement dangereux. Il faut pouvoir **discriminer la qualite** entre des BT valides (score 0.97 vs. 1.0) pour guider l'amelioration du dataset et du modele.

6.2 Decision

Validateur a **trois niveaux** implementes dans `validate_bt.py` :

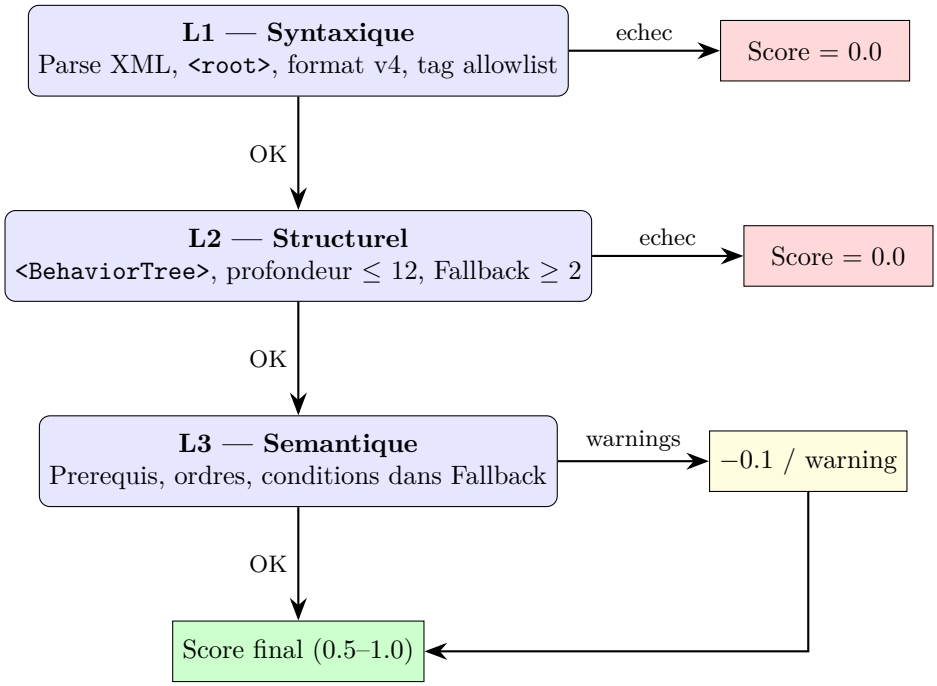

Figure 2 – Flux de validation a trois niveaux

6.3 Niveau L1 — Syntaxique (bloquant)

Ce niveau verifie la **forme** du XML. Tout echec entraine un score de **0.0** (BT rejete).

— **Parse XML** : le texte doit etre un XML bien forme (`xml.etree.ElementTree.fromstring`).

— **Tag racine** : doit etre `<root>` avec l'attribut `BTCPP_format="4"`.
— **Liste blanche de tags** : chaque tag du BT doit appartenir a l'ensemble `VALID_TAGS` (32 tags : 27 skills + `root`, `BehaviorTree`, `Sequence`, `Fallback`, `Parallel`).
— **`<MoveAndStop/>` obligatoire** : tout BT doit contenir ce skill (le robot doit toujours s'arreter a la fin de sa mission).

6.4 Niveau L2 — Structurel (bloquant)

Ce niveau verifie la **structure** de l'arbre. Tout echec entraine un score de **0.0**.

— `<BehaviorTree>` doit etre present sous `<root>`.
— Les noeuds de controle (`Sequence`, `Fallback`, `Parallel`) ne doivent **pas etre vides**.
— Profondeur maximale : **12** niveaux d'imbrication (`MAX_DEPTH = 12`).
— Nombre maximal de skills : **35** noeuds (`MAX_SKILLS = 35`).
— Un `<Fallback>` doit avoir au moins **2 branches** (sinon il n'a aucun sens — un Fallback a 1 branche est juste une indirection inutile).

6.5 Niveau L3 — Semantique (avertissements)

Ce niveau verifie la **logique metier** du BT. Les erreurs a ce niveau ne rejettent pas le BT mais reduisent le score de -0.1 par avertissement (plancher a 0.5).

— **Prerequis** : certains skills doivent apparaitre *avant* d'autres. Le dictionnaire `PREREQUISITES` definit 4 regles :

Skill	Doit etre precede de	Raison
`MissionStructureValid`	`LoadMission`	On ne peut valider une mission avant de la charger
`CreatePath`	`ProjectPointOnNetwork`	On ne peut creer un chemin sans projeter les points
`AgregatePath`	`CreatePath`	On ne peut agreger des chemins avant de les creer
`GenerateMissionSequence`	`LoadMission`	La sequence depend de la mission chargee

— **Premier skill** : doit etre `<LoadMission/>` (ou `<SimulationStarted/>` si la mission est une simulation).
— **Dernier skill** : doit etre `<MoveAndStop/>` (tolerance : `<SignalAndWaitForOrder/>` peut le suivre).
— **Conditions dans Fallback** : les skills de type *condition* (`MissionTerminated`, `MissionFullyTreated`, `MeasurementsQualityValidated`, `MeasurementsEnforcedValidated`) doivent etre a l'interieur d'un noeud `<Fallback>`. En effet, ces conditions servent de « garde » pour quitter une boucle — elles n'ont de sens que dans un contexte de branchement.

6.6 Protocole de test

L'evaluation utilise **15 missions de test** non presentes dans le jeu d'entrainement, reparties en trois niveaux de difficulte :

— **5 missions classiques** : navigation simple, inspection standard.

— **5 missions complexes / hors distribution** : simulation avec inspection corrective, navigation multi-segments avec mesures.
— **5 missions ambigues** : formulations volontairement imprecises pour tester la robustesse.

Meilleur resultat : sur le modele v4 scaled (2 000 exemples), les **15/15 missions** obtiennent un score de **1.0** avec **0 avertissement**.

Baseline zero-shot : sans fine-tuning, Mistral-7B produit **0/10 BT valides** (score 0.0) — le modele ne connait pas le format BehaviorTree.CPP v4 et hallucine des tags generiques.

References : `validate_bt.py`, `finetune_lora_xml.py` (lignes 339–422).

7 Tableau recapitulatif

Table 7 – Synthese des cinq decisions d'architecture

Axe	Decision	Alternative ecartee	Justification cle	Fichier
Proxy	Dataset synthetique (2 000 ex. a partir d'1 XML reel)	Annotation manuelle, donnees reelles	Cout prohibitif, BT compositionnels → generation par blocs	`generate_dataset_v4.py`
RAG	Rejete (injection statique)	RAG vectoriel, injection par regles, few-shot	27 skills = 270 tok = 7 % contexte, seuil > 50 non atteint	`RAG_ARCHITECTURE.md`
Securite	Defense en profondeur 3 couches	Validation simple (parse XML seul)	GBNF = 0 hallucination + validateur 3 niveaux = BT surs	`nav4rail_grammar.py`
Fine-tuning	QLoRA 4-bit NF4, Mistral-7B, 0.58 % params	Fine-tuning complet (∼56 Go), GPT-2, Llama-3	P100 16 Go, format `[INST]` natif	`finetune_lora_xml.py`
Evaluation	Validateur L1+L2+L3, score 0.0–1.0	Parse XML simple	Distingue erreurs syntaxiques / structurelles / semantiques	`validate_bt.py`

8 Catalogue des 27 skills NAV4RAIL

Pour reference, voici les 27 skills organises par famille :

References des fichiers

`finetune_lora_xml.py` Pipeline d'entrainement QLoRA et d'inference (496 lignes).

`validate_bt.py` Validateur de Behavior Trees a 3 niveaux (300 lignes).

`nav4rail_grammar.py` Grammaire GBNF et contrainte de decodage (198 lignes).

`generate_dataset_v4.py` Generateur du dataset synthetique v4 (2 000 exemples).

`RAG_ARCHITECTURE.md` Analyse detaillee du choix fine-tuning vs. RAG (266 lignes).

`SKILLS_CATALOG.md` Catalogue de reference des 27 skills NAV4RAIL.

`RESULTS.md` Resultats experimentaux detailles de toutes les versions.

`README.md` Documentation generale du pipeline.

`job_finetune.sh` / `job_finetune_v4.sh` Scripts SLURM d'orchestration sur le cluster.

Table 8 – Catalogue des skills NAV4RAIL par famille

Famille	**Skill**	**Type**
12*Preparation (12)	LoadMission	Action
	MissionStructureValid	Condition
	UpdateCurrentGeneratedActivity	Action
	ProjectPointOnNetwork	Action
	CreatePath	Action
	AgregatePath	Action
	MissionFullyTreated	Condition
	PassAdvancedPath	Action
	PassMission	Action
	GenerateMissionSequence	Action
	GenerateCorrectiveSubSequence	Action
	InsertCorrectiveSubSequence	Action
9*Motion (9)	MissionTerminated	Condition
	CheckCurrentStepType	Condition
	PassMotionParameters	Action
	Move	Action
	UpdateCurrentExecutedStep	Action
	Deccelerate	Action
	MoveAndStop	Action
	SignalAndWaitForOrder	Action
	IsRobotPoseProjectionActive	Condition
5*Inspection (5)	ManageMeasurements	Action
	AnalyseMeasurements	Action
	MeasurementsQualityValidated	Condition
	PassDefectsLocalization	Action
	MeasurementsEnforcedValidated	Condition
Simulation (1)	SimulationStarted	Condition